Programación en Java para la Algoritmos y estructuras de datos (AyEdD) Parte I

Rodrigo Pérez Rodríguez

El subconjunto del lenguaje Java que se utilizará en la asignatura se describe en dos documentos. En esta primera parte se introducen los elementos básicos del lenguaje Java que permiten realizar programas sin utilizar orientación a objetos, utilizando únicamente tipos primitivos, secuencias de caracteres, arrays, métodos estáticos y paquetes para entrada/salida.

Cualquier programa secuencial[1] puede escribirse únicamente con las siguientes sentencias elementales que están presentes en la mayoría de los lenguajes de programación de alto nivel (Fortran, COBOL, Python, Java, C, C++, Pascal, Ada, Lisp, JavaScript,...):

[1] Los programas secuenciales están compuestos de un único flujo de ejecución: en un instante determinado sólo se está ejecutando una de las sentencias del programa secuencial. Los programas concurrentes están compuestos de varios flujos de ejecución. En un programa concurrente en un instante determinado puede estarse ejecutando más de una sentencia a la vez si el hardware dispone de varios microprocesadores o de varios núcleos dentro del mismo microprocesador.

Declaraciones de datos en las que el programador da nombre a zonas de memoria en las que se almacenarán los datos que va a utilizar el programa como entrada o como salida de los algoritmos, o para almacenar datos intermedios generados por estos. Ejemplos:

```
int i;
```

```
double[] velocidad = {2.3, 4.45, 100.01};
```

Sentencias de asignación para guardar[2] el resultado de la evaluación de una expresión que contiene operaciones aritméticas y lógicas. Ejemplos:

[2] Puede guardarse un dato en la memoria principal o en un registro del microprocesador, según haya traducido el compilador o el intérprete del lenguaje la sentencia en instrucciones del microprocesador.

```
y = 3 * (x + 1);
```

```
y += 1;
```

Sentencias que alteran el flujo de ejecución secuencial. Las instrucciones cargadas en la memoria principal se ejecutan una a continuación de otra, secuencialmente, en el orden en el que están almacenadas. Para alterar este flujo secuencial, en los lenguajes de programación se utilizan sentencias a veces denominadas estructuras de control, de dos tipos: *alternativas* como `if` y `switch`, y *repetitivas* como `while` y `for`.

Estas sentencias son elementales porque sin ellas resultaría difícil escribir programas útiles, y porque sólo con ellas se puede escribir cualquier programa.

Para poder escribir programas más grandes, desde los años 50 del siglo pasado se fueron añadiendo a los lenguajes de programación otros mecanismos que complementan las sentencias elementales:

Los procedimientos y las funciones: sirven para poder expresar abstracciones funcionales reutilizables. Escribir una función o un procedimiento permite reutilizar sus sentencias mediante llamadas al código de la función o procedimiento. Las funciones y los procedimientos se identifican con un nombre, unos parámetros formales y un valor devuelto en el caso de las funciones. Los parámetros formales son variables cuyo ámbito es la función o procedimiento. Se crean cuando es invocada la función y desaparecen cuando ésta retorna. A veces se les denomina variables argumento.

La abstracción funcional ayuda al programador a organizar la complejidad del programa, creando nuevas abstracciones que le permiten abordar la programación de problemas mayores.

Las clases y los objetos: sirven para poder crear nuevos *tipos abstractos de datos*, encapsulando las variables que almacenan los valores junto a las funciones que operan sobre los mismos[3].

Los módulos: sirven para organizar el código, agrupando bajo un mismo nombre un conjunto de clases y/o funciones.

[3] A las funciones de un objeto se las suele denominar métodos.

A partir del código fuente, los programas denominados compiladores e intérpretes generan el código máquina o binario que contiene las instrucciones codificadas en binario que sabe ejecutar el microprocesador.

Estas instrucciones son sencillas, de una complejidad similar a la de las sentencias elementales de los lenguajes de alto nivel:

se evalúan expresiones aritméticas y lógicas

se copian datos en las asignaciones

se cambia el flujo de control

El resto de mecanismos (funciones, procedimientos, clases, módulos) permite a los humanos manejar la complejidad de programas grandes, pero no tienen reflejo en el código máquina generado por compiladores e intérpretes.

TANTO LAS INSTRUCCIONES DEL CÓDIGO MÁQUINA COMO LOS DATOS sobre los que estas operan tienen que estar almacenados en la memoria principal del ordenador.

Para que un programa comience a ejecutarse tiene que estar almacenada en la memoria principal al menos la primera de sus instrucciones.

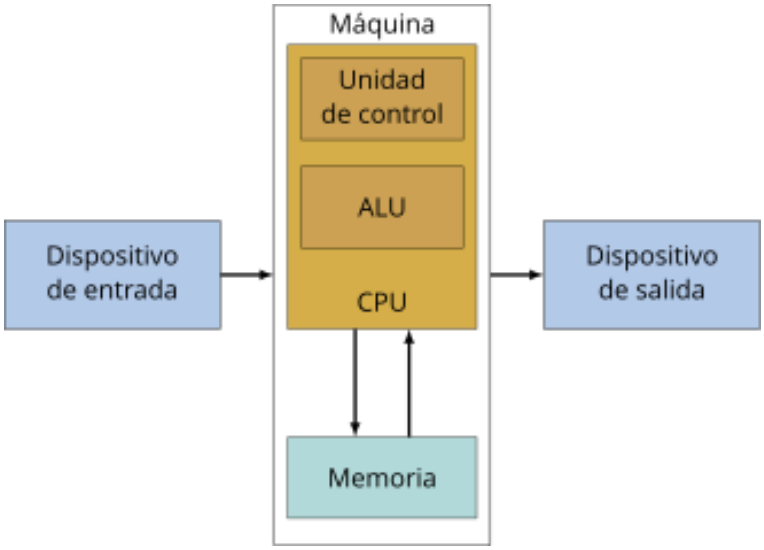
Cuando llega el momento de ejecutar una instrucción, los datos que utiliza la instrucción como operandos han de estar en la memoria principal.

Los datos resultado de la ejecución de una instrucción son igualmente almacenados en la memoria principal.

Prácticamente la totalidad de los ordenadores que usamos hoy día funcionan siguiendo este modo de funcionamiento en el que

instrucciones y datos están almacenados en la memoria principal.

A los ordenadores que funcionan de este modo se les denomina ordenadores de programa almacenado u ordenadores con arquitectura von Neumann.



ESTUDIAR ALGORITMOS Y ESTRUCTURAS DE DATOS utilizando un lenguaje de programación puede dificultar el percibir independientemente la idea del algoritmo de los detalles de su implementación en un lenguaje determinado.

Los objetivos principales de un lenguaje de programación son maximizar la eficiencia de la ejecución y facilitar la escritura de programas grandes.

Cuando se expresan algoritmos en un lenguaje de programación hay que escribir más texto que si se expresasen en pseudocódigo pues en el lenguaje de programación aparecen en el texto del programa anotaciones para declarar clases, objetos, tipos de datos en las declaraciones de variables y de los parámetros formales de las funciones.

Estos elementos son necesarios para escribir programas grandes de manera robusta pues los tipos ayudan a compiladores e intérpretes a detectar fallos cometidos por los programadores al expresar sus algoritmos antes de que estos se produzcan una vez el programa se ha arrancado.

Sin embargo, estas anotaciones no son necesarias para comprender los algoritmos.

Por eso en esta asignatura utilizaremos un subconjunto del lenguaje de programación Java que permite expresar algoritmos sin oscurecer su comprensión, y probarlos en programas robustos escritos en Java.

Lo aprendido con este subconjunto de Java es muy fácilmente trasladable a otros lenguajes de programación.

Este documento es una guía de estudio para este subconjunto del lenguaje que debes complementar con la lectura del libro de texto de la asignatura[4] .

[4] Lee las sección 1.1 del libro de texto: La biblioteca de la URJC da acceso a este libro: `https://algs4.cs.princeton.edu/11model/`

Utilizar un lenguaje de programación en lugar de pseudocódigo o el lenguaje de las matemáticas para expresar los algoritmos tiene ventajas e inconvenientes.

Estas son algunas ventajas:

Los programas permiten expresar no sólo de manera concisa y elegante, sino también de manera completa los algoritmos.

Se pueden ejecutar los programas para estudiar experimentalmente las propiedades de los algoritmos.

Se pueden utilizar los algoritmos inmediatamente en aplicaciones que los usen con algún propósito específico.

Llamamos Modelo de programación a la tripleta formada por:

(1) el subconjunto de Java que utilizaremos en la asignatura,

(2) las bibliotecas de software de la biblioteca estándar de Java, y las específicas de la asignatura, y

(3) la funcionalidad del sistema operativo en la que nos apoyaremos para escribir programas completos que implementen los algoritmos y las estructuras de datos que aprenderemos.

Herramientas y material de apoyo

Tenéis a vuestra disposición el fichero `algs4.jar` que, contiene las bibliotecas utilizadas en la asignatura y el fichero `algs4-data.zip`, con datos para probar los diferentes algoritmos.

Descárgalos y descomprime `algs4-data.zip` ejecutando el siguiente comando: `unzip algs4-data.zip`.

Comprueba que dentro del directorio descomprimido están los siguientes ficheros aparte de muchos otros:

`algs4-data/largeW.txt` y `algs4-data/largeT.txt`.

Estos dos ficheros contienen datos de entrada para programas de ejemplo que iremos estudiando, la mayoría de los cuales están disponibles en el repositorio del autor del manual de referencia:

`https://github.com/kevin-wayne/algs4`

Instalación del compilador Java en tu ordenador. La versión del compilador instalada en los laboratorios GNU/Linux de la ETSIT es la `17.0.5`.

Puedes comprobarlo ejecutando estos dos comandos que comprueban la versión del compilador `javac` y de la máquina virtual de Java `java`:

```
$ javac -version
javac 17.0.5

$ java -version
openjdk version "17.0.5" 2022-10-18
OpenJDK Runtime Environment (build 17.0.5+8-Ubuntu-2ubuntu122.04)
OpenJDK 64-Bit Server VM (build 17.0.5+8-Ubuntu-2ubuntu122.04, mixed mode, sharing)
```

Si tienes instalado en tu ordenador Ubuntu 20.04 o Ubuntu 22.04 puedes instalar esta misma versión ejecutando el siguiente comando:

```
$ sudo apt-get install openjdk-17-jdk
```

Instalación del IDE IntelliJ. En el laboratorio está instalado el IDE (*Integrated Development Environment*) IntelliJ IDEA.

Puedes instalarlo en tu propio ordenador utilizando la *Snap Store*.

Un programa en Java

A CONTINUACIÓN SE MUESTRA UN PROGRAMA JAVA COMPLETO que implementa una aplicación de filtrado de números utilizando una lista blanca[5].

[5] *Whitelist filtering*: dado un conjunto de números permitidos (lista blanca), se le van presentando al programa números. El programa muestra en la salida estándar aquellos números presentados que no estan en la lista blanca.

Un programa Java comienza ejecutando la primera sentencia del método `main()`.

Este método recibe el argumento `args` que es un array de `String`.

Cuando se arranca el programa se le pueden pasar argumentos al programa que estarán disponibles en `args` dentro del programa.

Para realizar la búsqueda de cada número en la lista blanca se utiliza el método `indexOf`, que va comparando el número que se le pasa como primer parámetro `key` con cada uno de los números del array `a` que se le pasa como segundo parámetro.

El método `indexOf` tiene dos sentencias `return` distintas.

En la línea 11 termina devolviendo el índice en el que se encuentra `key` en el array `a` cuando lo encuentra.

En la línea 12 termina devolviendo el valor `-1` cuando tras comparar `key` con todos los elementos de `a` no lo ha encontrado [6]:

[6] `-1` es un valor fuera del rango de los índices del array que se utiliza para indicar al método llamante `main` que no se ha encontrado el número `key` en el array `a`

```
1  import java.util.Arrays;
2  import edu.princeton.cs.algs4.In;
3  import edu.princeton.cs.algs4.StdIn;
4  import edu.princeton.cs.algs4.StdOut;
5
6  public class LinearSearch
7  {
8      public static int indexOf(int key, int[] a)
9      {
10         for (int i = 0; i < a.length; i++)
11             { if (a[i] == key) { return i; } }
12         return -1;
13     }
14     public static void main(String[] args)
15     {
16         In           in = new In(args[0]);
17         int[] whitelist = in.readAllInts();
18
19         while (!StdIn.isEmpty())
20         {
21             int key = StdIn.readInt();
22             if (indexOf(key, whitelist) == -1)
23                 { StdOut.println(key); }
24         }
```

```
25      }
26  }
```

Compilación y ejecución. Este programa ha de estar guardado en un fichero de nombre `LinearSearch.java`.

En el fichero `algs4-data/largeW.txt` hay 1 millón (10^6) de números, uno en cada línea del fichero, que utilizaremos como lista blanca (Whitelist) de números permitidos.

En el fichero `algs4-data/largeT.txt` hay 10 millones (10^7) de números que queremos usar como test para comprobar si están o no en la lista blanca.

El compilador de Java se puede ejecutar desde la línea de comandos:

```
% javac -cp .:algs4.jar LinearSearch.java
```

[7] En el laboratorio está copiado el fichero en `/opt/ayedd` por lo que puedes ejecutar el siguiente comando: `% javac -cp .:/opt/ayedd/algs4.jar LinearSearch.java`

La opción `-cp` significa *class path*.

Se utiliza para indicarle al compilador en qué directorios o ficheros de clases debe buscar clases Java que se necesiten para compilar el código de la clase `LinearSearch.java` que se está compilando.

En este caso le estamos indicando dos lugares en los que mirar: con el carácter `.` indicamos el directorio actual, desde el que estamos llamando al compilador, luego viene el carácter `:` utilizado como separador, y finalmente el fichero que contiene clases utilizadas en el curso, `algs4.jar`.

Una vez compilada la clase `LinearSearch.java`, si no hay errores, el compilador genera el fichero `LinearSearch.class` que contiene el *bytecode* del programa.

El *bytecode* no es el código máquina que puede ejecutar el microprocesador, sino un código compuesto de instrucciones para una máquina virtual de Java o *Java Virtual Machine (JVM)*.

El *bytecode* puede ejecutarse en cualquier ordenador que implemente una máquina virtual de Java, que no es más que un programa que sabe traducir el *bytecode* a código de un microprocesador real. Puedes comprobar el aspecto del *bytecode* de una clase Java ejecutando este comando: `javap -c -verbose LinearSearch.class`. Cuando se ejecuta la máquina virtual, ésta va traduciendo el *bytecode* del fichero `LinearSearch.class` a código máquina y ejecutándolo en el microprocesador. A las máquinas virtuales de Java que funcionan así se las denomina *Just in Time*. Otras implementaciones generan previamente a su ejecución el código máquina.

Para poder ejecutar el código de `LinearSearch.class` se necesita un programa que implemente una máquina virtual de Java para que traduzca el *bytecode* a código máquina del microprocesador concreto que estamos utilizando.

Así se ejecuta la clase llamando a la máquina virtual de Java con el comando `java`:

```
% java  -cp .:algs4.jar LinearSearch algs4-data/largeW.txt < algs4-data/largeT.txt
499569
984875
...
...
32449528
```

```
38862597
69830567
```

.

La salida producida por el programa son los números del fichero `largeT.txt` que no están en la lista blanca (fichero `largeW.txt`).

Estos números se mostrarán en la salida estándar.

Ejecutemos el programa paso a paso y analicemos su funcionamiento en este url: https://goo.gl/SCDdbb.

En este url está el programa, si bien la lista blanca no se lee de un fichero.

Los números a buscar en la lista blanca se le proporcionan por la entrada estándar (`stdin`) pero no 10^7 sino sólo 4.

Podemos comprobar el tiempo que tarda en ejecutarse el programa utilizando el comando `time`:

```
% time java -cp .:algs4.jar LinearSearch algs4-data/largeW.txt < algs4-data/largeT.txt
499569
984875
...
...
32449528
38862597
69830567

real 45m47,807s
user 45m37,932s
sys 0m5,003s
```

Medición del tiempo de ejecución del programa `LinearSearch.java`: más de 45 minutos y 47 segundos

`LinearSearch.java` tarda en el ordenador en el que lo hemos ejecutado 45 minutos y 47 segundos en buscar los 10^7 números del fichero `largeT.txt` en un array que contiene la lista blanca con los 10^6 números del fichero `largeW.txt`, es decir, es capaz de evaluar unos 3,700 por segundo.

Aprender a utilizar algoritmos y estructuras de datos eficientes (algoritmos que tarden poco tiempo en ejecutarse y que utilicen estructuras de datos que ocupen poco espacio en memoria) es uno de los objetivos principales de esta asignatura.

Cada vez que se quiere comprobar si un número del fichero `largeT.txt` está en la lista blanca lo vamos comparando en el método `indexOf` con todos los números de la lista blanca.

Este método utiliza un algoritmo de *búsqueda lineal*: cada número hay que buscarlo en `whitelist` comparándolo con todos los elementos del array hasta que aparezca o hasta que se llegue al final sin encontrarlo.

El programa (cliente) recibe un argumento en la línea de comandos, disponible en `args[0]` como el `String` `algs4-data/largeW.txt` ya que este String es el que se ha escrito justo después del nombre de la clase del programa. La entrada estándar se ha redirigido al arrancar el programa (`< algs4-data/largeT.txt`) por lo que cuando lea de la entrada estándar en realidad leerá del fichero `algs4-data/largeT.txt`

¿Podemos mejorar este programa?

Si ordenamos previamente el array `whitelist` podríamos utilizar este otro algoritmo para realizar la búsqueda en el método `indexOf(int key, int[] a)`:

> Recorrer el array:
>
> > Si se encuentra el número buscado se devuelve éste
> >
> > Si se encuentra uno mayor que el buscado se devuelve -1
> >
> > Si se llega al final sin encontrarlo se devuelve -1

Al estar ordenado, de esta forma nos ahorramos comparar con elementos mayores, pues es seguro que cuando comparemos con el primero que se mayor que el buscado, sabemos que ya no podrá estar en la lista blanca el número buscado, ya que todos serán aún mayores.

Copia ahora el fichero `LinearSearch.java` en otro llamado `SortedLinearSearch.java`, cambia el nombre a la clase, y añade esta línea para ordenar la lista blanca, justo antes del `while` en el método `main`:

```
Arrays.sort(whitelist);
```

Modifica el método `indexOf(int key, int[] a)` para que realice el algoritmo que hemos comentado más arriba.

Comprueba cuánto tiempo se ahorra así.

Compara los resultados[8].

Ten en cuenta que ahora tenemos el sobrecoste de tener que ordenar una vez al principio la lista blanca.

Aún así se consigue una mejora en el tiempo total de ejecución del programa.

Puedes ejecutarlo paso a paso en este url: `https://cscircles.cemc.uwaterloo.ca/java_visualize/`. Cuando escribas el código en *Java Visualizer* ten en cuenta que no son necesarios los `import` para las bibliotecas de la asignatura pues el servidor hace por nosotros un `import edu.princeton.cs.algs4.*;`.

Pero una vez que gastamos el tiempo en ordenar la lista blanca podemos utilizar otro algoritmo que reduce considerablemente el tiempo de ejecución.

Cuando buscas en un diccionario una palabra estás utilizando un algoritmo similar al implementado por el método `indexOf` en este otro programa, al que se denomina búsqueda binaria[9]:

[8] Puedes compartir tus resultados en el foro de la asignatura, poniendo el tiempo que tarda `LinearSearch` y el que tarda `SortedLinearSearch`

[9] Binary Search.

```
1  import edu.princeton.cs.algs4.In;
2  import edu.princeton.cs.algs4.StdIn;
3  import edu.princeton.cs.algs4.StdOut;
4
5  import java.util.Arrays;
6
7  public class BinarySearch {
8      public static int indexOf(int key, int[] a) {
9          int lo = 0;
10         int hi = a.length - 1;
11
12         while (lo <= hi) {
13             int mid = lo + (hi - lo) / 2;
14             if (key < a[mid]) {
15                 hi = mid - 1;
16             } else if (key > a[mid]) {
17                 lo = mid + 1;
18             } else {
19                 return mid;
20             }
21         }
22         return -1;
23     }
24
25     public static void main(String[] args) {
26         In in = new In(args[0]);
27         int[] whitelist = in.readAllInts();
28
29         Arrays.sort(whitelist);
30         while (!StdIn.isEmpty()) {
31             int key = StdIn.readInt();
32             if (indexOf(key, whitelist) == -1) {
33                 StdOut.println(key);
34             }
35         }
36     }
37 }
```

Con fondo gris aparecen los cambios respecto a `LinearSearch.java`: el método `indexOf` (líneas 8-21) utiliza el algoritmo de búsqueda binaria, que requiere que los elementos estén ordenados (línea 28).

El algoritmo de búsqueda binaria requiere que los elementos sobre los que se realiza la búsqueda estén ordenados, al igual que las palabras en un diccionario. Por ello en el código llamamos a `Arrays.sort` antes de llamar a `indexOf`.

Tiene un sobrecoste inicial al tener que ordenar la lista blanca, pero esta es una operación que sólo hay que realizar una vez al principio.

Ejecutemos el programa paso a paso y analicemos su funciona-

miento en este url: https://goo.gl/K4tpn7.

Veamos cuánto tarda el programa utilizando el algoritmo de búsqueda binaria:

```
% time java -cp .:algs4.jar BinarySearch algs4-data/largeW.txt < algs4-data/largeT.txt
499569
984875
...
...
32449528
38862597
69830567

real 0m14,890s
user 0m16,323s
sys 0m0,927s
```

Medición del tiempo de ejecución del programa `LinearSearch.java`

Medición del tiempo de ejecución del programa `BinarySearch.java`: ¡Poco más de 14 segundos!

En este caso sólo ha tardado 14,89 segundos. El tiempo de ejecución del programa se reduce más de dos órdenes de magnitud (se ha dividido por 10^2, pasando de más de 46 minutos a menos de 15 segundos para buscar los 10^7 números del fichero `largeT.txt` entre los 10^6 números de la lista blanca de números del fichero `largeW.txt`.

Pregunta: ¿Te ha resultado sorprendente? ¿A qué es debida esta reducción tan significativa del tiempo de ejecución?

Trata de entender el funcionamiento del algoritmo.

Piensa cuántas comparaciones se realizan en el caso peor (cuando no está el número en el array `whitelist`).

En `LinearSearch` había que comparar con todos los números del array.

¿Con cuántos hay que comparar ahora?

Gracias a este algoritmo, el número máximo de comparaciones que hay que hacer cada vez que se busca un número en `whitelist` es $log_2(whitelist.length)$.

Un algoritmo más adecuado para el problema que tratamos de resolver nos ha permitido realizar 666,666 búsquedas por segundo frente a las 3,700 que conseguíamos con `LinearSearch.java` en el mismo hardware.

Reflexiona sobre la importancia que tiene elegir los algoritmos y estructuras más adecuados para cada problema

Estructura de un programa Java

EN JAVA UN PROGRAMA ES UNA CLASE. El programa `BinarySearch.java` es una clase.

Cualquier código que se escribe en Java ha de estar dentro de una clase.

Una clase puede ser sólo una biblioteca de métodos estáticos (funciones), o puede definir un tipo abstracto de datos.

EN UNA CLASE SE PUEDEN UTILIZAR LOS SIGUIENTES 7 COMPONENTES:

(1) Tipos primitivos de datos y expresiones,

(2) Sentencias,

(3) Arrays,

(4) Métodos estáticos,

(5) Strings,

(6) Entrada/Salida,

(7) Abstracción de datos.

Tipos primitivos de datos y expresiones.

Un tipo de datos es un conjunto de valores y el conjunto de operaciones que operan con dichos valores.

El lenguaje Java tiene cuatro tipos primitivos de tipos de datos:

int: Números enteros con las operaciones aritméticas

double: Números reales con las operaciones aritméticas

boolean: Valores booleanos `true` y `false`, con las operaciones lógicas

char: Caracteres alfanuméricos y símbolos que se pueden teclear

En un programa Java se almacena la información en *variables*.

Las variables se nombran con *identificadores*. **Cada variable está asociada a un tipo de datos**[10] y almacena un valor de un tipo de datos.

[10] Recuerda que en Python el tipo no está asociado a la variable: se puede almacenar un valor de un tipo en una variable y más tarde almacenar un valor de otro tipo en esa misma variable. En Java no.

Se usan *expresiones* para aplicar las *operaciones* disponibles para cada tipo de datos.

La siguiente tabla recoge ejemplos y definiciones de varios de los términos que estamos definiendo:

término	ejemplos	definición
tipo primitivo de datos	`int double boolean char`	conjunto de valores y de operaciones sobre esos valores (forman parte del lenguaje Java)
identificador	`a abc Ab$ a_b ab123 lo`	secuencia de letras, dígitos, `_`, y `$`, siempre que el primero no sea un dígito
variable	[cualquier identificador]	nombra un valor de un tipo de datos
operador	`+ - * /`	nombra una operación de un tipo de datos

literal	`int` `1 0 -42`
	`double` `2.0 1.0e-15 3.14`
	`boolean` `true false`
	`char` `'a' '+' '9' '\n'`
		representación de un valor en el código fuente

expresión	`int` `lo + (hi - lo)/2`
	`double` `1.0e-15 * t`
	`boolean` `lo <= hi`
		un literal, una variable, o una secuencia de operaciones sobre literales y/o variables que produce un valor

En esta otra tabla se resumen los valores y operadores de los tipos primitivos de Java `int`, `double`, `boolean` y `char`:

tipo	conjunto de valores	operadores	expresiones típicas	
			expresión	**valor**

`int`	enteros entre -2^{31} y $2^{31} - 1$ (complemento a dos de 32 bits)	+ (suma)
		- (resta)
		* (multiplicación)
		/ (división)
		% (resto)
			`5 + 3`
			`5 - 3`
			`5 * 3`
			`5 / 3`
			`5 % 3`
				`8`
				`2`
				`15`
				`1`
				`2`

`double`	números reales de precisión doble (estándar IEEE 754 de 64 bits)	+ (suma)
		- (resta)
		* (multiplicación)
		/ (división)
			`3.141 - .03`
			`2.0 - 2.0e-7`
			`100 * .015`
			`6.02e23 / 2.0`
				`3.111`
				`1.9999998`
				`1.5`
				`3.01e23`

`boolean`	`true` o `false`	&& (and)
		|| (or)
		! (not)
		^ (xor)
			`true && false`
			`false || true`
			`!false`
			`true ^ true`
				`false`
				`true`
				`true`
				`false`

`char`	caracteres (16 bits)	[*operaciones aritméticas, no se suelen utilizar*]		

Expresiones. Las expresiones utilizan notación infija: *operando* operador *operando*.

Cuando una expresión contiene más de un operador el orden de precedencia especifica el orden en el que se aplican los operadores.

Los operadores *, / y % tienen mayor precedencia (se aplican antes) que + y -; entre los operadores lógicos, ! es el de mayor precedencia, después |&&| y después ||.

Los operadores de la misma precedencia se asocian por la izquierda (se aplican de izquierda a derecha).

Con paréntesis se puede alterar el orden de aplicación de los operandos definido por las anteriores reglas.

Conversión de tipos. Los números se promocionan automáticamente a un tipo más inclusivo si no se pierde información.

Por ejemplo, en la expresión 1 + 2.5, el 1 se promociona al valor double 1.0 y la expresión se evalúa al valor double 3.5.

Un cast es una directiva explícita para convertir un valor de un tipo a uno de otro tipo.

Por ejemplo, (int) 3.7 es 3.

Cuando se hace *casting* de un double a un int se trunca hacia cero (se elimina la parte fraccionaria).

Comparaciones. Los siguientes operadores de diversos tipos comparan dos valores del mismo tipo y producen un valor booleano:

==	igual
!=	no igual
<	menor que
<=	menor que o igual
>	mayor que
>=	mayor que o igual

Otros tipos primitivos de datos.

Los valores del tipo primitivo de Java `int` se representan con 32 bits y los valores del tipo `double` con 64 bits.

Java tiene 5 tipos primitivos adicionales que se representan con otras cantidades de bits:

`long`	enteros de 64 bits, con operaciones aritméticas
`short`	enteros de 16 bits, con operaciones aritméticas
`char`	caracteres de 16 bits, con operaciones aritméticas
`byte`	enteros de 8 bits, con operaciones aritméticas
`float`	números reales de 32 bits en representación de coma flotante con precisión simple, con operaciones aritméticas

Sentencias.

Un programa Java está compuesto de sentencias que definen la computación, creando y manipulando variables, asignando valores del tipo adecuado a las mismas, y controlando el flujo de ejecución de dichas sentencias.

Un *bloque de sentencias* es un conjunto de una o más sentencias expresadas entre llaves.

Las declaraciones crean variables del tipo especificado y las nombran con identificadores. Java es un lenguaje fuertemente tipado, lo que significa que el compilador de Java comprueba la consistencia.

El ámbito de una variable es la parte del programa en la que la variable se haya definida.

Las asignaciones asignan un valor de un tipo de datos (definido por una expresión) a una variable.

Las declaraciones con inicialización combinan una declaración con una asignación para inicializar una variable nada más declararla.

Asignaciones implícitas. Los siguientes atajos permiten asignar un nuevo valor a una variable calculado en función de su valor actual:

Operadores de incremento/decremento: `i++` provoca el mismo efecto sobre la variable `i` que `i = i + 1`, e `i--` provoca el mismo efecto que `i = i - 1`. En la expresión en la que aparecen estos atajos se substituye `i++` / `i--` por el valor de `i`, y después se realiza el incremento/decremento.

También existen los atajos `++i` / `--i`, que incrementan/decrementan el valor de `i` en una unidad. La diferencia está en que ahora, primero se realiza el incremento/decremento, y luego se substituye en la expresión `++i` / `--i` por el valor ya incrementado/decrementado de `i`

Otros operadores compuestoss: `i /= 2` es lo mismo que `i = i/2` y `i *= 2` es lo mismo que `i = i*2`. En ambos casos el operador se aplica al valor de `i` después de utilizarse su valor en la expresión en la que aparece el atajo.

Las condicionales permiten alterar el flujo de ejecución de las sentencias en función de una condición expresada mediante una expresión booleana. Si la éxpresión booleana de la condición se evalúa a `true` se ejecuta un bloque de sentencias:

Se puede especificar un bloque de sentencias que se ejecuta si no se cumple la condición:

Los bucles `while` permiten alterar más profundamente el flujo de ejecución de las sentencias. Mientras la condición del bucle `while` se evalúe a `true` se ejecutará el bloque de sentencias especificado, al que denominamos cuerpo del bucle.

Dentro del cuerpo de un bucle `while` se pueden utilizar dos sentencias adicionales: `break` y `continue`:

`continue` hace que se interrumpa la ejecución de la iteración actual del cuerpo del bucle, ejecutánose a continuación una nueva iteración de las sentencias del cuerpo del bucle

`break` hace que se interrumpa completamante la ejecución del cuerpo del bucle, ejecutándose a continuación la sentencia siguiente al bucle

Los bucles `for` son una alternativa para código que sigue este patrón: (1) inicializar una variable índice con algún valor, (2) usar un bucle `while` para comprobar una condición en la que interviene dicha variable índice, siendo la última sentencia del cuerpo del bucle un incremento/decremento de la variable índice.

El siguiente bucle `for` es equivalente al bucle `while` que aparece más abajo:

Bucle `while` equivalente:

Los bloques de una sóla sentencia se pueden expresar sin llaves en la rama de una sentencia condicional o en el cuerpo de un bucle.

La siguiente tabla muestra ejemplos de sentencias Java:

sentencia	ejemplos	definición
declaración	`int i;` `double c;`	crea una variable de un tipo específico, y la nombra con un identificador
asignación	`a = b + 3;` `discriminant = b*b - 4.0*c;`	asigna un valor de un tipo de datos a una variable
declaración con inicialización	`int i = 1;` `double c = 3.141592625;`	declaración que también asigna un valor inicial
asignación implícita	`i++;` `i += 1;`	`i = i + 1;`
condicional (if)	`if (x < 0) x = -x;`	
condicional (if-else)	`if (x > y) max = x;` `else max = y;`	ejecuta una sentencia, dependiendo de la expresión booleana

bucle (while)	`int v = 1;`
	`while (v <= N)`
	`  v = 2*v;`
	
	`double t = c;`
	`while (Math.abs(t - c/t) > 1e-15*t)`
	`  t = (c/t + t) / 2.0;`
		ejecuta sentencias hasta que la expresión booleana sea `false`

bucle (for)	`for (int i = 1; i <= N; i++)`
	`  sum += 1.0/i;`
	`for (int i = 0; i <= N; i++)`
	`  StdOut.println(2*Math.PI*i/N);`
		versión compacta de un bucle `while`

llamada a método	`int key = StdIn.readInt();`	invoca otros métodos
return	`return false;`	termina la ejecución de un método devolviendo un valor

Arrays.

Un array almacena una secuencia de valores, todos ellos del mismo tipo. Si tenemos `N` valores se puede usar la notación `a[i]` para referirnos al valor que ocupa la posición `i` para cualquier valor de `i` comprendido entre `0` y `N-1`.

Construcción de arrays. Hay tres pasos en la construcción de un array:

Declaración del identificador y el tipo de los valores que almacenará el array

Creación del espacio en memoria que ocuparán los valores del array

Inicialización de los valores del array. Hay tres formas de hacerlo:

1. En este caso, declaramos la referencia a un array, luego se reserva memoria y con `for` se dan valores:

2. En este caso declaramos y creamos memoria en la misma sentencia:

3. Aquí, sin declarar el tamaño pero sí los valores en la inicialización. Se crea memoria implícitamente y el tamaño es el de los valores entre llaves:

Uso de un array. Una vez que se ha creado un array, su tamaño queda fijado. La longitud de un array `a[]` se obtiene con `a.length`. Java realiza comprobación automática de los límites: si se intenta acceder a una posición de un array con un índice que no existe se eleva la excepción `ArrayIndexOutOfBoundsException`[11]

[11] Una excepción es un error que ocurre en tiempo de ejecución. Se gestionar o tratar las excepciones para que el programa no termine cuando se produce una.

Aliasing. El identificador de una variable que contiene un array referencia a todo el array: si se asigna una variable que contiene un array a otra, lo que se copia es la referencia, por lo que ambas variables referencian un único array en memoria. Veamos un ejemplo:

La última asignación a `b[i]` hace que también `a[i]` sea 5678 pues `a` y `b` referencian el mismo array. Esta situación se conoce como aliasing[12] y puede propiciar errores difíciles de detectar.

[12] El array `b` es un alias del array `a`.

Arrays de dos dimensiones. Un array de dos dimensiones en Java es un array de una dimensión cuyos elementos son arrays de una dimensión. Cada uno de los arrays puede ser de tamaños diferentes si bien es frecuente que sean del mismo tamaño. Para referirse a un elemento en la fila `i` y la columna `j` de un array de dos dimensiones se utiliza la notación `a[i][j]`.

Métodos estáticos.

A los métodos estáticos se los denomina funciones en muchos lenguajes de programación pues se comportan como funciones matemáticas (relación en la que cada valor en la entrada se asocia con un valor en la salida). Cada método estático en Java es una secuencia de sentencias que se ejecutan una tras otra cuando el método estático es llamado.

Cada método estático está compuesto de una declaración o firma (*signature*) en la que se especifica su nombre, el tipo del valor que devuelve y la lista entre paréntesis de los parámetros formales[13], con sus tipos, y un cuerpo.

[13] También conocidos como variables de los argumentos o variables argumento.

Invocación de un método estático. Se llama a un método estático con su nombre, seguido por expresiones que especifican los valores de los parámetros, separados por comas y entre paréntesis. Cuando se llama a un método sus parámetros formales[14] se inicializan con los correspondientes valores de las expresiones usadas en la llamada, a los que denominamos parámetros actuales. Una sentencia `return` termina un método estático, devolviendo el control a la sentencia desde la que se llamó al método estático. Si el método estático devuelve un valor, éste se especifica en una expresión que se pone entre paréntesis como argumento a la sentencia `return`.

[14] También conocidos como variables de los argumentos o variables argumento.

Propiedades de los métodos.

Los parámetros se pasan por valor. Cuando se llama a un método estático, se evalúa la expresión de cada parámetro actual que se usa en la llamada y el valor resultante se copia en el parámetro formal[15]. A esto se le denomina paso por valor. Por tanto las modificaciones que se realicen a los parámetros formales en el método sólo afectan a las zonas de memoria ocupadas por los parámetros formales que contienen copias, y no a las que se hayan utilizado como parámetros actuales en el código que llamó al método estático.

[15] También llamado variable argumento

Los arrays (y los objetos) también se pasan por valor. Pero recuerda que un array es una referencia, por lo que se pasa una copia de la referencia al parámetro formal. Por tanto, dentro del método estático no se puede modificar la referencia. Pero a través de ella sí se pueden modificar los valores de los elementos del array referenciado (o el valor del objeto).

Los nombres de los métodos pueden sobrecargarse. Los métodos de una clase pueden tener el mismo nombre, siempre que su declaración o firma (*signature*) sea distinta. A esta característica se le denomina sobrecarga (*overloading*).

Un método devuelve un único valor pero puede tener múltiples sentencias `return`*.* Cuando se ejecuta una de sus sentencias `return` el método termina, devolviéndose un único valor, del tipo especificado en la declaración o firma.

Un método puede tener efectos laterales. Un método puede usar la palabra reservada `void` como el tipo del valor devuelto para indicar que no devuelve ningún valor. Un método así puede producir efectos laterales (consumir valores de la entrada estándar o de otros ficheros, producir salidas (pantalla, red, ficheros), cambiar valores en variables del programa, etc.).

Recursión. Un método recursivo es uno en el que entre sus sentencias se hace una llamada a él mismo, directa o indirectamente a través de una llamada a un segundo método dentro del cuál se llama al primero. Cuando se utiliza la recursión hay que tener tener en cuenta estos aspectos:

Es importante que la recursión tenga un caso base en el que se interrumpan las llamadas recursivas.

Las llamadas recursivas deben realizarse para resolver subproblemas que de algún modo sean *más pequeños*, de forma que se acabe convergiendo al caso base para que las llamadas recursivas dejen de realizarse en algún momento.

Modelo básico de programación. Una biblioteca de métodos estáticos es un conjunto de métodos estáticos que se definen en una clase Java. El modelo básico de programación en Java consiste en desarrollar un programa que resuelve cierta tarea creando una biblioteca de métodos estáticos, uno de los cuáles en `main()`.

Programación modular. Las bibliotecas o módulos de métodos estáticos permiten realizar una programación modular en la que los métodos estáticos de una biblioteca pueden invocar a los métodos estáticos definidos en otras bibliotecas. Las ventajas de esta técnica son varias:

Los módulos pueden tener un tamaño razonable, no muy grande

Se puede compartir y reutilizar código sin tener que reimplementarlo

Se pueden realizar mejoras substituyendo implementaciones fácilmente

Se pueden desarrollar modelos abstractos adecuados a la resolución de problemas de programación concretos

Se hace más fácil la detección de fallos en el código

Referencias